

CORSO NODE.JS + EXPRESS + JWT

Middleware avanzati e persistenza con SQLite

Dispensa del corso

Giorno 2 — Parte 1

DOCENTE

Massimo Petrolati

Product Manager · Namirial SpA

Edizione 2026 · Documento di studio per studenti livello base

Indice

Cosa troverai in questa dispensa

1	Approfondimento middleware	6
1.1	Da dove ripartiamo	6
1.2	Application-level vs router-level	6
1.3	Un middleware utile: il logger di durata	7
1.4	Middleware con argomenti (factory pattern)	8
1.5	L'ordine di registrazione conta (parecchio)	9
2	Error handler centralizzato	12
2.1	Il problema da risolvere	12
2.2	Il middleware di error handling	12
2.3	Una classe di errore custom: <code>AppError</code>	13
2.4	<code>Async/await</code> e <code>next(err)</code>	14
3	Introduzione a SQLite con <code>sqlite3</code> + <code>sqlite</code>	18
3.1	Perché SQLite per il corso	18
3.2	Quale driver SQLite per Node	18
3.3	Installazione	19
3.4	Le tre operazioni base	19
3.5	Aprire il <code>.db</code> con un viewer grafico	21
4	Schema della tabella <code>users</code>	23
4.1	La DDL completa	23
4.2	Una colonna alla volta, e perché	23
4.3	Bootstrap: eseguire la DDL all'avvio	24
5	Il Repository Pattern	27
5.1	Il principio	27
5.2	Scrivere il <code>userRepository.js</code>	27
5.3	Refactor: dal CRUD in-memory al repository	29
5.4	Esporre solo quello che serve	30
6	Struttura a layer completa	32
6.1	Dove siamo arrivati	32
6.2	Cosa ogni layer NON deve sapere	32
6.3	Perché tutta questa cerimonia	33

Introduzione

Dal CRUD giocattolo a un'applicazione vera

Bentornato al Giorno 2. Ieri abbiamo costruito un piccolo CRUD di utenti in memoria con Express, organizzato in router modulari. Funzionava, ma con tre limiti grossi che oggi affrontiamo uno alla volta.

Primo: gli **errori** sono gestiti in modo inconsistente, ogni controller fa di testa sua e quando qualcosa va storto il client riceve risposte diverse a seconda di chi ha scritto la route. Vedremo come centralizzare la gestione errori con un solo middleware in fondo alla pipeline.

Secondo: i **middleware** li abbiamo introdotti come funzioni `(req, res, next)`, ma non abbiamo davvero scavato. Oggi ne vediamo tre tipi diversi, in che ordine vanno registrati e perché l'ordine conta.

Terzo, il più grosso: i dati **vivono in un array**. Se riavviamo il server, addio. È ora di mettere un vero database sotto. Sceglieremo **SQLite** con il driver `sqlite3` e il piccolo wrapper `sqlite` per avere un'API a Promise/async-await: zero configurazione, file singolo sul disco, sintassi SQL standard, e niente compilatori C++ da installare sui PC Windows.

Cosa costruiamo concretamente oggi

A fine giornata avremo:

- Un **error handler centralizzato** che cattura qualunque `next(err)` dai controller e restituisce risposte JSON uniformi.
- Una classe `AppError` per costruire errori con `statusCode` e `message` senza ripeterci.
- Una tabella `users` su SQLite, vera, con vincolo `UNIQUE` sull'email e indice per la lookup.
- Un file `userRepository.js` che è **l'unico punto di contatto col DB**: se domani cambiamo SQLite con Postgres, l'unico file che riscriviamo è questo.
- Il CRUD di ieri completamente rifattorizzato per usare il repository invece dell'array.

Come è strutturata

La dispensa segue i sei blocchi della lezione:

- Approfondimento middleware: i tre tipi, ordine di esecuzione, scrittura di middleware utili.
- Error handler centralizzato: pattern `next(err)`, classe `AppError`, wrapper per async/await.
- Introduzione a SQLite con `sqlite3` + `sqlite` (driver con binari precompilati per Windows +

wrapper a Promise).

- Schema della tabella `users`: DDL, scelte di progetto, indice sull'email.
- Il **Repository Pattern**: cosa è, perché ci interessa, come si scrive.
- Struttura a layer completa che useremo da qui al progetto finale.

I box che incontrerai

- **CODICE**: riferimento al mini-progetto pronto nella cartella `codice-esempi/giorno-2-parte-1/`.
- **Suggerimento**: consiglio pratico per evitare problemi tipici.
- **Attenzione**: trappola comune o cosa NON fare.

Come imparare davvero

Oggi vedrai per la prima volta un **pattern architetturale** (il repository) e un **principio di design** (separazione delle responsabilità). Non leggerli passivamente: a metà giornata fermati e prova a spiegarti ad alta voce *perché* il `userRepository` esiste, e cosa cambia rispetto al farei le query direttamente nel controller. Se non sai rispondere, rileggi. Sono i mattoncini che useremo ogni giorno per il resto della tua carriera.

Dove trovi il codice

Tutti gli esempi della dispensa sono pronti nel **repository GitHub del corso**:

github.com/mpetrolati/2026-corso-backend-nodejs-express-its

Se non l'hai già clonato ieri, fallo ora:

```
$ git clone https://github.com/mpetrolati/2026-corso-backend-nodejs-express-its.git
$ cd 2026-corso-backend-nodejs-express-its
```

I file di questa dispensa stanno nella cartella `codice-esempi/giorno-2-parte-1/`.

CAPITOLO 01

Approfondimento middle-ware

I tre tipi, l'ordine di esecuzione, come si scrivono quelli utili

1. Approfondimento middleware

1.1 Da dove ripartiamo

Ieri abbiamo definito un middleware in una riga: una funzione `(req, res, next)` che fa qualcosa e poi chiama `next()` per passare il controllo al successivo. Va bene per cominciare, ma è una semplificazione. Express distingue in realtà tre famiglie di middleware, ognuna con un suo ruolo:

- **Application-level middleware** — registrato direttamente sull'app con `app.use(...)`. Si applica a tutte le richieste (o a tutte quelle sotto un certo path).
- **Router-level middleware** — registrato su un router specifico con `router.use(...)`. Si applica solo alle route di quel router.
- **Error-handling middleware** — ha una firma diversa, a 4 parametri `(err, req, res, next)`, e viene eseguito **solo** quando qualche middleware precedente fa `next(err)`.

I primi due li abbiamo già usati ieri. L'error-handling lo vediamo nel capitolo 2: è il cuore di un'API REST seria.

1.2 Application-level vs router-level

La differenza è di **scope**: dove si applicano. Ricordi il middleware `morgan` di ieri?

```
import express from 'express';
import morgan from 'morgan';
import userRoutes from './routes/userRoutes.js';

const app = express();

// APPLICATION-LEVEL: vale per TUTTE le richieste
app.use(morgan('dev'));
app.use(express.json());

// ROUTER-LEVEL: vale solo per le richieste sotto /api/users
app.use('/api/users', userRoutes);

export default app;
```

Dentro `userRoutes.js`, possiamo registrare ulteriori middleware che si applicheranno solo a quel router:

```
// src/routes/userRoutes.js
import { Router } from 'express';

const router = Router();

// ROUTER-LEVEL: vale solo per le route di questo router
router.use((req, res, next) => {
  console.log(`[utenti] ${req.method} ${req.url}`);
  next();
});

router.get('/', (req, res) => res.json([/* ... */]));

export default router;
```

In altre parole: se vuoi un comportamento globale (logging, parsing del body, CORS, auth-su-tutta-l'app) usi `app.use`. Se vuoi un comportamento specifico di una sezione (es. "solo le route admin richiedono ruolo admin") usi `router.use` dentro il router admin.

Regola pratica

Quando ti chiedi "dove metto questo middleware?", chiediti: **si applica a tutta l'app o solo a un sotto-insieme di route?**. La risposta ti dice subito se va in `app.js` (application-level) o nel router specifico (router-level).

1.3 Un middleware utile: il logger di durata

Scriviamone uno con un caso d'uso vero. Obiettivo: **loggere metodo, path, status code e tempo di risposta** di ogni richiesta. Esiste già `morgan`, ma scriverlo a mano una volta aiuta a capirlo davvero.

```
// src/middlewares/durata.js
export function durata(req, res, next) {
  const inizio = Date.now();

  // L'evento 'finish' di res scatta quando la risposta
  // e' stata mandata al client.
  res.on('finish', () => {
    const ms = Date.now() - inizio;
    console.log(`${req.method} ${req.url} -> ${res.statusCode} (${ms}ms)`);
  });

  next(); // <-- non bloccare la pipeline
}
```

Tre cose interessanti:

- Salviamo `Date.now()` nello scope della funzione middleware. Ogni richiesta ha il suo `inizio`, perché Express chiama la funzione una volta per richiesta.
- Ci agganciamo all'evento `finish` di `res` (è un `EventEmitter`, ricordi il Giorno 1?). Scatta quando la risposta è stata interamente mandata al client.
- Chiamiamo `next()` **subito**, prima di registrare il listener? No, l'ordine non importa qui — ma non chiamiamo `next()` dentro la callback di `finish`, perché a quel punto la pipeline è già finita.

In `app.js`:

```
import { durata } from './middlewares/durata.js';

app.use(durata);
```

Output che vedrai nel terminale:

```
GET /api/users      -> 200   (3ms)
POST /api/users     -> 201   (18ms)
GET /api/users/999  -> 404   (1ms)
```

1.4 Middleware con argomenti (factory pattern)

Quando un middleware ha bisogno di **configurazione**, lo si scrive come una **factory**: una funzione che restituisce il middleware vero e proprio. È un pattern che useremo letteralmente domani per il middleware `verifyToken(req, res, next)` con `secret` e algoritmo configurabili.

```
// src/middlewares/richiediHeader.js
export function richiediHeader(nome) {
  return function (req, res, next) {
    if (!req.headers[nome.toLowerCase()]) {
      return res.status(400).json({ errore: `Manca header ${nome}` });
    }
    next();
  };
}
```

E lo usi così:

```
app.use('/api', richiediHeader('X-Client-Id'));
// Tutte le route sotto /api richiedono l'header X-Client-Id.
```

Pattern che useremo sempre

Il middleware factory è la base del middleware di autenticazione del Giorno 3: `verifyToken(secret)` ritornerà un middleware che legge l'header `Authorization`, verifica la firma con quel secret, popola `req.user` e chiama `next()`. Stessa identica struttura di `richiediHeader` qui sopra.

1.5 L'ordine di registrazione conta (parecchio)

Express esegue i middleware **nell'ordine esatto** in cui sono registrati con `app.use`. Sembra ovvio, ma è la causa numero uno di “ma perché non funziona?”. Tre regole pratiche da tenere sempre a mente:

- I middleware di **parsing** (`express.json`, `express.urlencoded`, `cors`, `helmet`) vanno **in cima**, prima delle route. Altrimenti `req.body` è `undefined` dentro la route.
- I middleware di **autenticazione** vanno **prima delle route che proteggono**. Sembra ovvio detto così, ma è facile dimenticarsene quando l'app cresce.
- Il middleware di **gestione errori** (4 parametri) va **ultimo**, dopo tutte le route. È un tema così importante che gli dedichiamo il prossimo capitolo intero.

```
const app = express();

app.use(express.json());           // 1) parsing body
app.use(durata);                   // 2) logger
app.use('/api', autentica);        // 3) auth su /api/*
app.use('/api/users', userRoutes); // 4) route
app.use(errorHandler);             // 5) error handler IN FONDO
```

CODICE: `giorno-2-parte-1/01-middleware-tipi/`

CAPITOLO 02

Error handler centralizzato

Un solo middleware che cattura tutti gli errori della tua API

2. Error handler centralizzato

2.1 Il problema da risolvere

Riguarda il CRUD di ieri. Ogni controller gestisce gli errori a modo suo:

```
router.get('/:id', (req, res) => {
  const user = store.findById(req.params.id);
  if (!user) return res.status(404).json({ errore: 'Non trovato' });
  res.json(user);
});

router.post('/', (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ errore: 'name e email obbligatori' });
  }
  // ...
});
```

Funziona, ma con dieci route ti ritrovi con dieci varianti di errore JSON. Una scrive `{errore: ...}`, un'altra `{message: ...}`, una terza ti restituisce solo testo. Il client non sa mai cosa aspettarsi.

L'altro problema, più sottile: cosa succede se dentro un controller una funzione asincrona **lancia un'eccezione** che non hai catturato? Esempio: una query al DB fallisce. Senza un error handler, Express manda al client una pagina HTML di default "Internal Server Error". Brutto da vedere, peggio da debuggare.

La soluzione è **centralizzare** la gestione errori in un solo posto.

2.2 Il middleware di error handling

Express riconosce un middleware come "error handler" dalla **firma a 4 parametri**. È l'unico caso in cui il numero di argomenti conta: con `(err, req, res, next)` è un error handler, con `(req, res, next)` è un middleware normale.

```
// src/middlewares/errorHandler.js
export function errorHandler(err, req, res, next) {
  console.error(err); // log per il docente / sviluppatore

  const status = err.statusCode || 500;
  const message = err.message || 'Errore interno';

  res.status(status).json({ errore: message });
}
```

Registralo per ultimo in `app.js`, dopo tutte le route:

```
app.use('/api/users', userRoutes);

// catch-all 404 per route inesistenti
app.use((req, res) => {
  res.status(404).json({ errore: 'Route non trovata' });
});

// error handler: cattura qualsiasi next(err)
app.use(errorHandler);

export default app;
```

Da ora i controller possono “rilanciare” gli errori al middleware centrale con `next(err)` invece di gestirli a mano:

```
router.get('/:id', (req, res, next) => {
  const user = store.findById(req.params.id);
  if (!user) {
    const err = new Error('Utente non trovato');
    err.statusCode = 404;
    return next(err); // <-- delego all'error handler
  }
  res.json(user);
});
```

Risultato: una sola scrittura di risposta JSON, un solo formato di errore per tutto il client. Pulito.

2.3 Una classe di errore custom: `AppError`

Creare l'oggetto errore a mano ogni volta (`const err = new Error(...); err.statusCode = ...`) è verboso. Ce lo costruiamo una volta sola in una classe:

```
// src/utils/AppError.js
export class AppError extends Error {
  constructor(message, statusCode = 500) {
    super(message);
    this.statusCode = statusCode;
    this.name = 'AppError';
  }
}
```

E nei controller diventa una riga:

```
import { AppError } from '../utils/AppError.js';

router.get('/:id', (req, res, next) => {
  const user = store.findById(req.params.id);
  if (!user) return next(new AppError('Utente non trovato', 404));
  res.json(user);
});

router.post('/', (req, res, next) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return next(new AppError('name e email sono obbligatori', 400));
  }
  // ... (vedremo i validatori veri nel pomeriggio)
});
```

Perché una classe e non un oggetto

Una classe ti dà un nome al tipo di errore: `instanceof AppError` ti dice se è un errore “previsto” (uno che hai pensato e codificato) o un’eccezione fuori dal tuo controllo (es. un crash di Node). Più avanti potrai estendere `AppError` in `ValidationError`, `AuthError`, ecc., con behavior specifico.

2.4 Async/await e `next(err)`

Quando il controller è `async`, una promise rifiutata non viene catturata automaticamente da Express. Devi farlo a mano con un `try/catch`:

```
router.get('/:id', async (req, res, next) => {
  try {
    const user = await store.findById(req.params.id);
    if (!user) return next(new AppError('Non trovato', 404));
    res.json(user);
  } catch (err) {
    next(err); // <-- delega l'eccezione all'error handler
  }
});
```

Funziona, ma il `try/catch` si ripete in ogni controller. Esistono due soluzioni standard.

Soluzione 1: wrapper `asyncHandler`

Un piccolo helper che wrappa il controller e cattura le promise rifiutate automaticamente:

```
// src/utils/asyncHandler.js
export const asyncHandler = (fn) => (req, res, next) => {
  Promise.resolve(fn(req, res, next)).catch(next);
};
```

E nei controller:

```
import { asyncHandler } from '../utils/asyncHandler.js';

router.get('/:id', asyncHandler(async (req, res) => {
  const user = await store.findById(req.params.id);
  if (!user) throw new AppError('Non trovato', 404);
  res.json(user);
}));
```

Niente più `try/catch`, niente più `next(err)` esplicito. Se la promise rifiuta, `.catch(next)` la cattura per te.

Soluzione 2: `express-async-errors`

Una libreria che fa la stessa cosa ma a livello globale: la importi una volta in `app.js` e da quel momento ogni `async` controller cattura automaticamente.

```
$ npm install express-async-errors
```

```
// src/app.js
import 'express-async-errors'; // <-- una sola volta, in cima

// poi tutti i controller async funzionano senza wrapper
```

Quale scegliere

Per progetti piccoli (il nostro corso): `asyncHandler` scritto a mano. È trasparente, niente magia, una funzione di 3 righe che hai sotto controllo.

Per progetti grossi: `express-async-errors` fa risparmiare un sacco di `asyncHandler(...)` ripetuti. È una dipendenza in più, ma stabile.

Nel Giorno 3 (Express 5 sta arrivando) la cattura automatica delle promise rifiutate diventerà nativa. Per ora restiamo su `asyncHandler`.

CODICE: `giorno-2-parte-1/02-error-handler/`

CAPITOLO 03

Introduzione a SQLite con `sqlite3` + `sqlite`

Un database vero, senza configurazione e in un file solo

3. Introduzione a SQLite con `sqlite3` + `sqlite`

3.1 Perché SQLite per il corso

Avremmo potuto scegliere PostgreSQL, MySQL o MongoDB. Abbiamo scelto SQLite per quattro ragioni didattiche, che vanno bene per qualunque corso e per parecchi progetti reali piccoli.

- **Zero configurazione:** non c'è un servizio da installare, un demone da avviare, un utente da creare. È una libreria che scrive su un file. Si lavora.
- **Tutto in un file:** la tua base dati è un singolo `.db` sul disco. Lo copi, lo committi (no, non committarlo), lo apri con un viewer grafico. Niente magie.
- **SQL standard:** la sintassi che imparerai qui vale per Postgres e MySQL al 90%. Migrare quando il progetto cresce è banale.
- **Usato in produzione vera:** tutto il sistema iOS lo usa per dati locali, Chrome, Firefox, Android. Non è un giocattolo: è il database più diffuso al mondo, per numero di installazioni.

3.2 Quale driver SQLite per Node

Esistono diversi driver SQLite per Node. I tre più comuni:

- `sqlite3`: il driver storico. Ships con **binari precompilati per Windows, macOS e Linux**, quindi `npm install` non richiede compilatori sul tuo PC. API nativa basata su callback.
- `better-sqlite3`: API sincrona, molto veloce. Ma è scritto in C++ e **richiede compilazione** (su Windows servono i “Visual Studio Build Tools”). Comodo se sei su macOS/Linux, doloroso su Windows.
- `node:sqlite`: modulo built-in di Node 22+. Promettente, ma ancora sperimentale (richiede il flag `--experimental-sqlite`).

Useremo `sqlite3` per il driver, perché “funziona e basta” su Windows senza scaricare nulla in più. Per evitare la verbosità delle callback, lo combiniamo con il piccolo wrapper `sqlite`, che espone la stessa libreria con un'API **Promise / async-await**. Risultato: due righe di setup, codice quasi identico a quello che faremmo in altre tecnologie.

```
// sqlite3 grezzo (callback, da non usare nel corso)
db.get('SELECT * FROM users WHERE id = ?', [1], (err, row) => {
  if (err) /* ... */;
  console.log(row);
});

// sqlite3 + sqlite (la nostra scelta): async/await pulito
const row = await db.get('SELECT * FROM users WHERE id = ?', 1);
console.log(row);
```

Async ovunque, ma non spaventa

Ogni operazione sul DB ritorna una `Promise`: dovrai mettere `await` davanti a ogni `db.get/.all/.run`, e le funzioni del repository diventeranno tutte `async`. È esattamente lo stesso pattern di `bcrypt.hash` che useremo nel pomeriggio. Tutto il flusso di Express è già asincrono: l'`asyncHandler` che abbiamo costruito nel capitolo 2 gestisce per noi qualunque promise rifiutata.

3.3 Installazione

```
$ npm install sqlite3 sqlite
```

Due pacchetti: `sqlite3` è il driver (con i binari precompilati), `sqlite` è il wrapper a Promise sopra di esso. Pochi KB in tutto, e installa pulito su qualunque sistema operativo senza chiedere compilatori.

3.4 Le tre operazioni base

La quasi totalità del lavoro si fa con tre metodi: `db.get` per leggere una riga, `db.all` per leggerle tutte, `db.run` per scrivere (INSERT / UPDATE / DELETE).

```
import sqlite3 from 'sqlite3';
import { open } from 'sqlite';

const db = await open({
  filename: './data/app.db',
  driver: sqlite3.Database
});

// .get(...) ritorna UNA riga o undefined
const user = await db.get('SELECT * FROM users WHERE id = ?', 1);

// .all(...) ritorna TUTTE le righe come array
const users = await db.all('SELECT * FROM users');

// .run(...) esegue INSERT / UPDATE / DELETE
const info = await db.run(
  'INSERT INTO users (email, password_hash) VALUES (?, ?)',
  'mario@example.com', 'hash...'
);

console.log('Inserito id:', info.lastID);
console.log('Righe affette:', info.changes);
```

Due dettagli importanti del wrapper `sqlite`:

- `open` è la funzione di setup. Ritorna una Promise: usiamo `await` al top-level del modulo `connection.js`, possibile grazie agli ES Modules.
- Il campo dell'ultimo id inserito si chiama `info.lastID` (better-sqlite3 lo chiama `lastInsertRowid`: piccola differenza da ricordare se vieni da quel mondo).

Query parametrizzate: il ? è obbligatorio

MAI concatenare le variabili nella query SQL. È la causa numero uno di vulnerabilità di sicurezza (SQL injection).

```
// SBAGLIATISSIMO -- non fare MAI questo
const email = req.body.email; // l'utente potrebbe mandare '' OR '1'='1"
await db.get('SELECT * FROM users WHERE email = '${email}');

// GIUSTO -- la libreria sanitizza i valori per te
await db.get('SELECT * FROM users WHERE email = ?', email);
```

I `?` sono **placeholder**: la libreria li sostituisce in modo sicuro coi valori che passi come argomenti aggiuntivi a `.get/.all/.run`. Imparalo come riflesso: ogni valore che viene dall'utente passa

SEMPRE attraverso un placeholder.

3.5 Aprire il .db con un viewer grafico

Durante il debug è utilissimo **guardare dentro** al file SQLite. Lo strumento gratuito più diffuso è **DB Browser for SQLite** (sqlitebrowser.org): installalo, apri il tuo `app.db` e vedrai tabelle, righe, indici in interfaccia grafica.

Effetto wow in aula

Quando in classe creiamo il primo utente via `POST /api/auth/register`, aprire DB Browser e vedere comparire la riga in tempo reale è il momento “ah, funziona davvero”. Stessa cosa varrà per te a casa: usa DB Browser ogni volta che vuoi capire se la tua query ha fatto quello che pensi.

CODICE: `giorno-2-parte-1/03-sqlite-base/`

CAPITOLO 04

Schema della tabella users

DDL, scelte di progetto e bootstrap del database

4. Schema della tabella `users`

4.1 La DDL completa

Ecco la tabella che useremo per tutto il resto del corso. Pensata per essere semplice oggi e estendibile domani:

```
CREATE TABLE IF NOT EXISTS users (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  email       TEXT NOT NULL UNIQUE,  
  password_hash TEXT NOT NULL,  
  name        TEXT,  
  role        TEXT NOT NULL DEFAULT 'user',  
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);
```

Ogni riga è una scelta. Vediamole una alla volta.

4.2 Una colonna alla volta, e perché

`id INTEGER PRIMARY KEY AUTOINCREMENT`

Una chiave primaria numerica auto-incrementante. In SQLite, `INTEGER PRIMARY KEY` è già auto-incrementante di suo — `AUTOINCREMENT` aggiunge la garanzia che gli id non vengano mai **riutilizzati** dopo una cancellazione. Per un'API REST è la scelta più semplice. In progetti più grossi si usano UUID, ma ne ripariamo un'altra volta.

`email TEXT NOT NULL UNIQUE`

L'email è il nostro identificativo logico dell'utente. `NOT NULL` perché senza email non c'è registrazione. `UNIQUE` perché non vogliamo due account con la stessa email: lo facciamo rispettare a **livello DB** cosicché anche se il check applicativo fallisce, il database rifiuta l'inserimento. "Trust the DB to be your last line of defense".

`password_hash TEXT NOT NULL`

Il nome del campo è una scelta deliberata: `password_hash`, non `password`. Serve come promemoria visivo a chiunque legga il codice o il DB: **qui dentro non c'è una password in chiaro, c'è un hash**. È un piccolo dettaglio che salva da un sacco di errori. Sull'hashing vero (`bcrypt`) torneremo

questo pomeriggio.

name TEXT

Il nome visualizzato. Niente `NOT NULL`: lo lasciamo opzionale, perché l'utente potrebbe registrarsi senza darcelo subito.

role TEXT NOT NULL DEFAULT 'user'

Un campo **predisposto** per la gestione permessi futura. Oggi non lo usiamo ancora: ogni utente è 'user'. Domani potremmo introdurre 'admin' e middleware che leggono `req.user.role`. Costa zero metterlo ora, costa molto aggiungerlo dopo (servirebbe una migrazione del DB).

created_at e **updated_at**

Due timestamp standard, valori di default `CURRENT_TIMESTAMP`. Servono per audit, per ordinare gli utenti per data di registrazione, per debugging. Sono **always-on**, in ogni tabella che fai nella tua carriera: prendili come abitudine.

L'indice su `email`

```
CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);
```

Tecnicamente non sarebbe necessario: `UNIQUE` su `email` crea già un indice implicito in SQLite. Ma è una **buona abitudine** dichiararlo esplicito: rende chiaro a chiunque legga lo schema che le query del tipo `WHERE email = ?` sono ottimizzate. E ne faremo tante: ogni login, ogni controllo di duplicato in registrazione.

4.3 Bootstrap: eseguire la DDL all'avvio

Vogliamo che il database venga creato (se non esiste) ogni volta che lanciamo l'app. Usiamo `IF NOT EXISTS` sia per la tabella che per l'indice, così l'operazione è **idempotente**: la possiamo rieseguire all'infinito senza danno.

Struttura dei file che useremo:

```
src/└─
  db/
    └─ connection.js ← apre il file SQLite
    └─ schema.sql    ← DDL della tabella users
```

```
// src/db/connection.js
import sqlite3 from 'sqlite3';
import { open } from 'sqlite';
import { readFileSync, mkdirSync } from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';

const __dirname = path.dirname(fileURLToPath(import.meta.url));

// 1. Mi assicuro che la cartella data/ esista
const dataDir = path.join(__dirname, '../data');
mkdirSync(dataDir, { recursive: true });

// 2. Apro (o creo) il file SQLite – top-level await!
const db = await open({
  filename: path.join(dataDir, 'app.db'),
  driver: sqlite3.Database
});

// 3. Eseguo la DDL al primo avvio (idempotente grazie a IF NOT EXISTS)
const schema = readFileSync(path.join(__dirname, 'schema.sql'), 'utf-8');
await db.exec(schema);

export default db;
```

Tre cose:

- `db.exec(...)` esegue uno script SQL multiplo (utile per la DDL che contiene più statement separati da `;`).
- Stiamo usando **top-level await** (`await open(...)` fuori da qualunque funzione async). È una feature che ESM ci regala: il modulo `connection.js` aspetta che il DB sia pronto prima di esportare `db`. Chiunque lo importi riceverà la connessione già aperta.
- `mkdirSync(dataDir, { recursive: true })` crea la cartella `data/` se non esiste: piccolo passo di bootstrap che evita errori al primo avvio.

Non committare il `.db`

Aggiungi `data/` (o `app.db`) al `.gitignore`: il database degli utenti non si committa su Git. Stesso discorso per il `.env` con i segreti, che vedremo nel pomeriggio.

CODICE: `giorno-2-parte-1/04-schema-users/`

CAPITOLO 05

Il Repository Pattern

Tutto il DB in un file solo: domani cambiare database costa cinque minuti

5. Il Repository Pattern

5.1 Il principio

Il database lo conosce solo il **repository**. Le route, i controller, i service non sanno se sotto c'è SQLite, Postgres, un file JSON o un foglio Excel. Chiamano funzioni come `findByEmail(email)` o `create({email, passwordHash})` senza preoccuparsi di cosa succede sotto al cofano.

Il vantaggio è enorme e si vede su due fronti:

- **Manutenibilità**: il giorno che cambiamo DB, l'unico file che riscriviamo è il repository. Tutti gli altri restano intatti.
- **Testabilità**: in test possiamo sostituire il repository con una versione finta (un *mock*) che ritorna dati hard-coded, senza dover toccare un database vero.

Questo non è “inventato” da Express o da Node: è un **pattern architetturale** (Repository Pattern) che esiste in qualsiasi linguaggio. Lo trovi in Spring, in .NET, in Django. Capirlo qui significa portarsi a casa un mattoncino che ti seguirà nella carriera.

5.2 Scrivere il `userRepository.js`

Cominciamo dalle quattro funzioni essenziali per la nostra Auth API: cercare per email, cercare per id, creare un utente, aggiornare.

```
// src/repositories/userRepository.js
import db from '../db/connection.js';

export async function findByEmail(email) {
  return db.get('SELECT * FROM users WHERE email = ?', email);
}

export async function findById(id) {
  return db.get('SELECT * FROM users WHERE id = ?', id);
}

export async function create({ email, passwordHash, name = null, role = 'user' }) {
  const info = await db.run(
    `INSERT INTO users (email, password_hash, name, role)
    VALUES (?, ?, ?, ?)`,
    email, passwordHash, name, role
  );

  // restituisco l'utente appena creato leggendolo dal DB
  return findById(info.lastID);
}

export async function update(id, data) {
  // costruisco la SET parte solo coi campi presenti
  const fields = [];
  const values = [];

  if (data.name !== undefined) { fields.push('name = ?'); values.push(data.name); }
  if (data.passwordHash) { fields.push('password_hash = ?'); values.push(data.passwordHash); }
  if (data.role) { fields.push('role = ?'); values.push(data.role); }
  }

  if (fields.length === 0) return findById(id); // niente da aggiornare

  // aggiorno anche updated_at
  fields.push("updated_at = CURRENT_TIMESTAMP");
  values.push(id);

  await db.run(`UPDATE users SET ${fields.join(', ')} WHERE id = ?`, ...values);
  return findById(id);
}

export async function deleteById(id) {
  const info = await db.run('DELETE FROM users WHERE id = ?', id);
  return info.changes > 0;
}
```

Sei cose da notare bene:

- Tutte le funzioni del repository sono `async` e ritornano Promise. Chi le chiama dovrà mettere `await` davanti.
- Tutte le query usano **placeholder** `?`. Nessuna concatenazione di stringhe SQL, mai.
- In `create` usiamo i **parametri nominati** (`{ email, passwordHash, name, role }`) come destrutturazione del primo argomento. Più chiaro a chi legge.
- `info.lastID` è il nome del campo nel wrapper `sqlite` per ottenere l'id dell'ultimo inserimento (in `better-sqlite3` si chiamava `lastInsertRowid` — piccola differenza).
- Il repository **non valida i dati**. Quello lo fa il layer sopra (services o validators). Il repository scrive ciò che gli viene chiesto.
- Il repository **non hasha la password**. Riceve già un `passwordHash`. L'hashing è logica di business, vive nel service. Su `bcrypt` torneremo questo pomeriggio.

5.3 Refactor: dal CRUD in-memory al repository

Il CRUD che abbiamo costruito ieri usava un finto store in memoria (`userStore.js`). Vediamo come cambiano le route quando il sotto-strato diventa il repository SQLite.

Prima (Giorno 1, in-memory)

```
import * as store from '../data/userStore.js';

router.get('/:id', (req, res) => {
  const user = store.findById(req.params.id);
  if (!user) return res.status(404).json({ errore: 'Non trovato' });
  res.json(user);
});
```

Dopo (Giorno 2, SQLite via repository)

```
import * as userRepository from '../repositories/userRepository.js';
import { AppError } from '../utils/AppError.js';
import { asyncHandler } from '../utils/asyncHandler.js';

router.get('/:id', asyncHandler(async (req, res) => {
  const user = await userRepository.findById(req.params.id);
  if (!user) throw new AppError('Utente non trovato', 404);
  res.json(user);
}));
```

La route resta molto simile, ma ora è `async` (la wrappiamo con `asyncHandler`, costruito nel capitolo 2) e mette `await` davanti alla chiamata del repository. Tutto il lavoro vero — aprire la connessione, eseguire SQL, leggere la riga, mapparla in un oggetto JS — avviene nel repository, fuori dalla vista del controller.

Il messaggio chiave

“Se domani cambiamo SQLite con PostgreSQL, l'unico file che riscrivo è `userRepository.js`. Se cambiamo Express con Fastify, l'unica cosa che cambia sono i controller e le route, mai i repository.”

Memorizzalo. È la promessa del pattern, ed è perché esiste.

5.4 Esporre solo quello che serve

Un dubbio frequente: il repository ritorna l'oggetto utente con **tutti** i campi del DB, incluso `password_hash`. Va bene esporlo al client tramite la route `GET /users/:id`? **No, assolutamente no.**

La rimozione del campo sensibile spetta al **layer sopra**: il controller o il service che chiama il repository. Esempio:

```
router.get('/:id', asyncHandler(async (req, res) => {
  const user = await userRepository.findById(req.params.id);
  if (!user) throw new AppError('Utente non trovato', 404);

  // tolgo password_hash prima di mandare al client
  const { password_hash, ...safeUser } = user;
  res.json(safeUser);
}));
```

Nel pomeriggio metteremo in piedi un piccolo `userService` che lo fa di default per tutte le route. Lo principio: **il repository ti dà tutto, sta a chi lo usa decidere cosa è sicuro mostrare.**

CODICE: `giorno-2-parte-1/05-user-repository/`

CAPITOLO 06

Struttura a layer completa

Riepilogo e filosofia di cosa ogni layer NON deve sapere

6. Struttura a layer completa

6.1 Dove siamo arrivati

Dopo questo capitolo, la struttura del progetto è quella che useremo **fino alla fine del corso**, con piccole aggiunte nei giorni successivi:

```
src/|—
db/|
  |— connection.js      ← apertura del DB + bootstrap schema|
  |— schema.sql         ← DDL della tabella users|—
repositories/|
  |— userRepository.js   ← UNICO punto di contatto col DB|—
routes/|
  |— userRoutes.js       ← /api/users/*|—
middlewares/|
  |— errorHandler.js     ← gestione errori centralizzata|—
utils/|
  |— AppError.js         ← classe di errore custom|
  |— asyncHandler.js     ← wrapper per route async|—
app.js               ← configurazione Express|—
server.js            ← entry point
```

Dal pomeriggio aggiungeremo `validators/` e `services/`. Dal Giorno 3 arriveranno `middlewares/authMiddleware.js`, `utils/hashPassword.js`, `utils/generateToken.js`.

6.2 Cosa ogni layer NON deve sapere

È la tabella più importante che porti a casa oggi. Memorizzala, ti tornerà utile in tutti i progetti backend della tua carriera.

Tabella delle responsabilità

Routes — conoscono URL e mappano i metodi HTTP ai controller. NON conoscono la logica di business né il DB.

Controllers — leggono `req`, chiamano i service, scrivono `res`. NON conoscono come funziona il DB.

Services — contengono la logica di business (hash password, generazione token, regole). NON conoscono Express (niente `req/res`).

Repositories — eseguono le query SQL. NON sanno che esistono JWT, bcrypt o controller.

Middlewares — cross-cutting concerns: auth, validation, errori, logging. NON contengono logica di business specifica del dominio.

6.3 Perché tutta questa cerimonia

Visto da fuori, sembra una struttura sovradimensionata per un'API di autenticazione semplice. Tre risposte rapide al “ma non basterebbe un `app.js` unico?”.

- **Cresce bene.** Quando aggiungeremo route per i post, per i commenti, per le notifiche, la struttura è già pronta: nuova route, nuovo controller, nuovo service, nuovo repository, fatto. Senza struttura, ogni nuova feature è un refactor.
- **Testabile.** Ogni layer si testa in isolamento. Il `userService.register()` si testa senza Express e senza DB, mockando il repository. Il `userRepository` si testa con un DB SQLite in memoria. Le route si testano con `supertest`, mockando i service.
- **È quello che troverai in azienda.** Tutti i progetti backend seri usano una variante di questo. Imparare la struttura qui significa essere produttivo dal primo giorno di lavoro.

Trappola del “troppo presto”

Non confondere “struttura a layer” con “sovra-ingegnerizzazione”. La nostra struttura ha 5 cartelle in tutto, ognuna con una responsabilità chiara. Non aggiungeremo **DI containers**, **interfacce astratte**, **factory di factory**, eccetera. Quello è da progetti enterprise giganti, ed è una conversazione da fare con anni di esperienza. Per ora, semplicità + disciplina.

Appendice

Cosa porti a casa e dove andiamo nel pomeriggio

Cosa ti porti a casa da questa lezione

- I tre tipi di middleware (application-level, router-level, error-handling) e perché l'ordine di registrazione conta.
- Un error handler centralizzato con classe `AppError` e wrapper `asyncHandler` per le route async.
- SQLite con `sqlite3` + `sqlite`: `db.get/.all/.run` con `async/await`, query parametrizzate, file unico, viewer grafico.
- Lo schema della tabella `users`: vincolo `UNIQUE` su email, `password_hash`, ruolo predisposto, timestamps, indice.
- Il Repository Pattern: il DB lo conosce solo il repository, tutto il resto chiama funzioni neutre.
- Una struttura a layer chiara, con un'idea precisa di cosa ogni layer NON deve sapere.

Cartella codice di riferimento

Tutti gli esempi sono pronti nella cartella `codice-esempi/giorno-2-parte-1/` del repository GitHub del corso:

github.com/mpetrolati/2026-corso-backend-nodejs-express-its

Se non l'hai ancora fatto, clonalo (o scaricalo come ZIP da GitHub):

```
$ git clone https://github.com/mpetrolati/2026-corso-backend-nodejs-express-its.git
$ cd 2026-corso-backend-nodejs-express-its/codice-esempi/giorno-2-parte-1
```

Ogni sottocartella è un mini-progetto autonomo con `package.json` e `README.md`:

- `01-middleware-tipi/` application-level, router-level, factory
- `02-error-handler/` `AppError`, error handler, `asyncHandler`
- `03-sqlite-base/` primi passi con `sqlite3` + `sqlite`
- `04-schema-users/` `schema.sql` + bootstrap della connessione
- `05-user-repository/` il repository completo (find, create, update, delete)

- `06-crud-sqlite/` il CRUD del Giorno 1 rifattorizzato su SQLite via repository

Cosa faremo nel pomeriggio

La Parte 2 del Giorno 2 monta sopra a tutto questo l'autenticazione vera:

- **Validazione input** con `express-validator` e un middleware `validate` centralizzato.
- **Variabili d'ambiente** con `dotenv`, file `.env` e `.env.example`.
- **Hashing password** con `bcrypt`: salt, work factor, perché MAI in chiaro.
- **Service layer**: dove vive la logica di business (`authService.register`).
- Implementazione end-to-end di `POST /api/auth/register` con validazione, hash, scrittura su DB.

Per approfondire

- **Express middleware ufficiali**: expressjs.com/en/guide/using-middleware.html.
- **sqlite (promise wrapper)**: github.com/kriasoft/node-sqlite — API completa con esempi, leggibile in 10 minuti.
- **sqlite3 (driver)**: github.com/TryGhost/node-sqlite3 — documentazione del driver sottostante, raramente serve.
- **SQLite syntax**: sqlite.org/lang.html — riferimento della sintassi SQL supportata.
- **DB Browser for SQLite**: sqlitebrowser.org — per ispezionare il file `.db`.
- **Repository Pattern, contesto storico**: cerca “Eric Evans Domain-Driven Design” — il libro è impegnativo ma il capitolo sui repository è didatticamente illuminante.

Un consiglio prima della pausa

Prima di tornare in aula, prova a riscrivere a memoria la firma delle 5 funzioni del `userRepository`: `findByEmail`, `findById`, `create`, `update`, `deleteById`. Solo i **nomi** e i **parametri**, non l'implementazione. Se non le ricordi tutte, riapri il capitolo 5: sono i mattoncini di tutto il pomeriggio.